

Experiment 4 : Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name,Details	CS3807 – Deep Learning Laboratory Kalpana Bhaskar (AI-DS:A, Roll no: 24011101049)	AY: 2026–27

1 Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2 Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3 Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4 Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5 LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6 AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7 VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8 Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9 GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy et al. in 2014, introduced the Inception Module, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10 ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

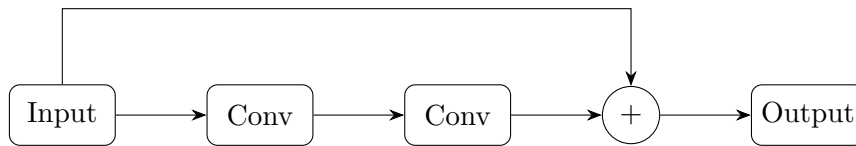


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11 Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters. The dilation rate determines the spacing between kernel elements. For a standard convolution, $D = 1$. For dilated convolution, $D > 1$, where zeros indicate inserted gaps.

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

Applications

- Semantic segmentation

- Medical image analysis
- Satellite imagery
- Object localization

12 Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13 Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

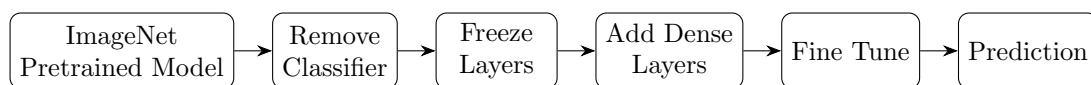


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14 Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Number of Classes: 10
- Image Size: $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15 Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet 50
- MobileNet V2
- Efficient Net B0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

Task 3: Model Training

Train the model using the following training parameters:

Parameter	Value
Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10–20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5-10 epochs.
3. Compare the classification accuracy before and after fine tuning.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

16 Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

17 Source Code

The complete source code, implementation details, and execution notebooks for this experiment are available at the following repository:

<https://github.com/your-username/your-repo-name>

18 Mandatory Plots

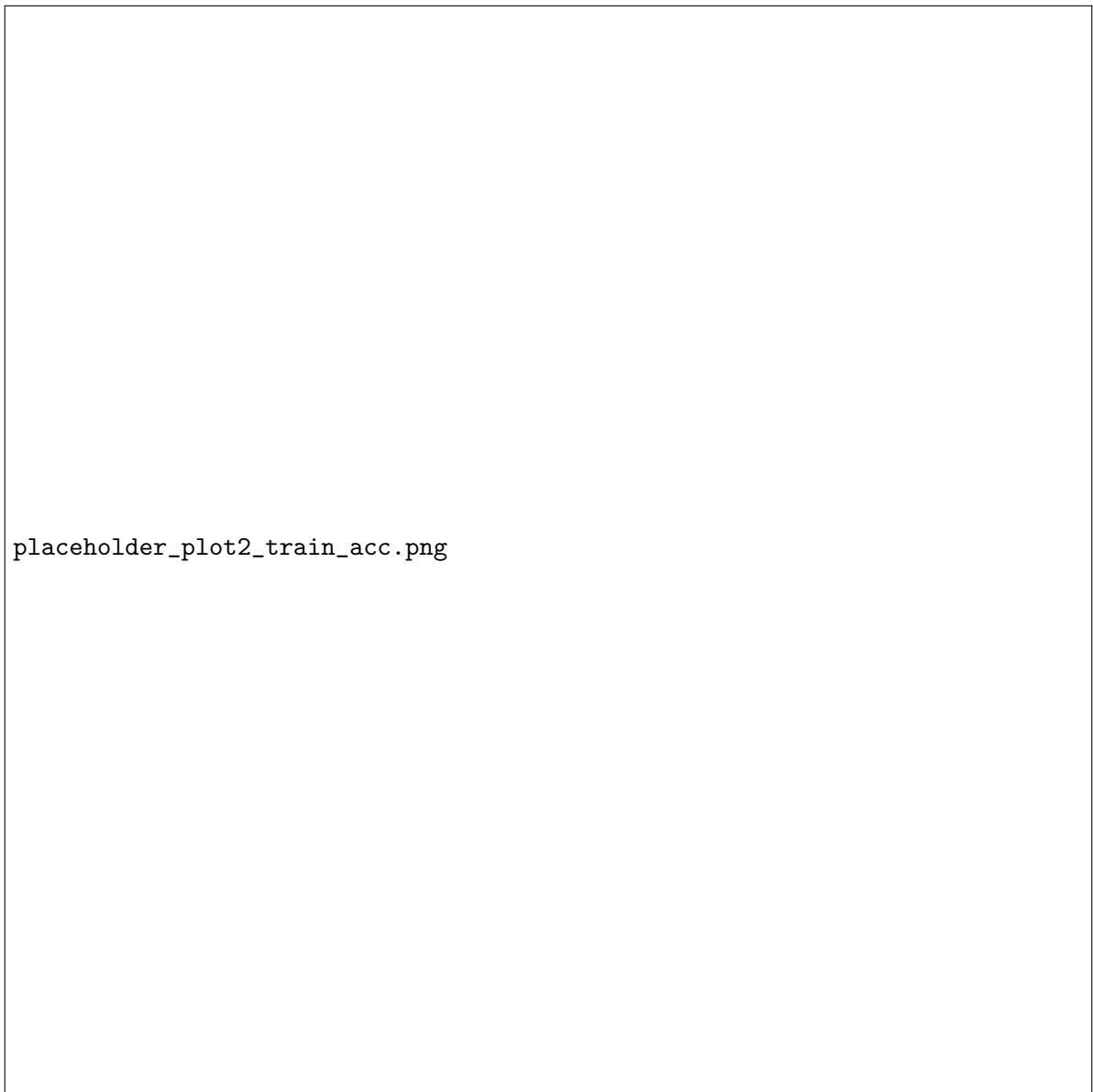
1. Sample CIFAR-10 Images



Figure 3: Sample CIFAR-10 images across different classes.

Interpretation: [Add your 2-3 line inference here]

2. Training Accuracy

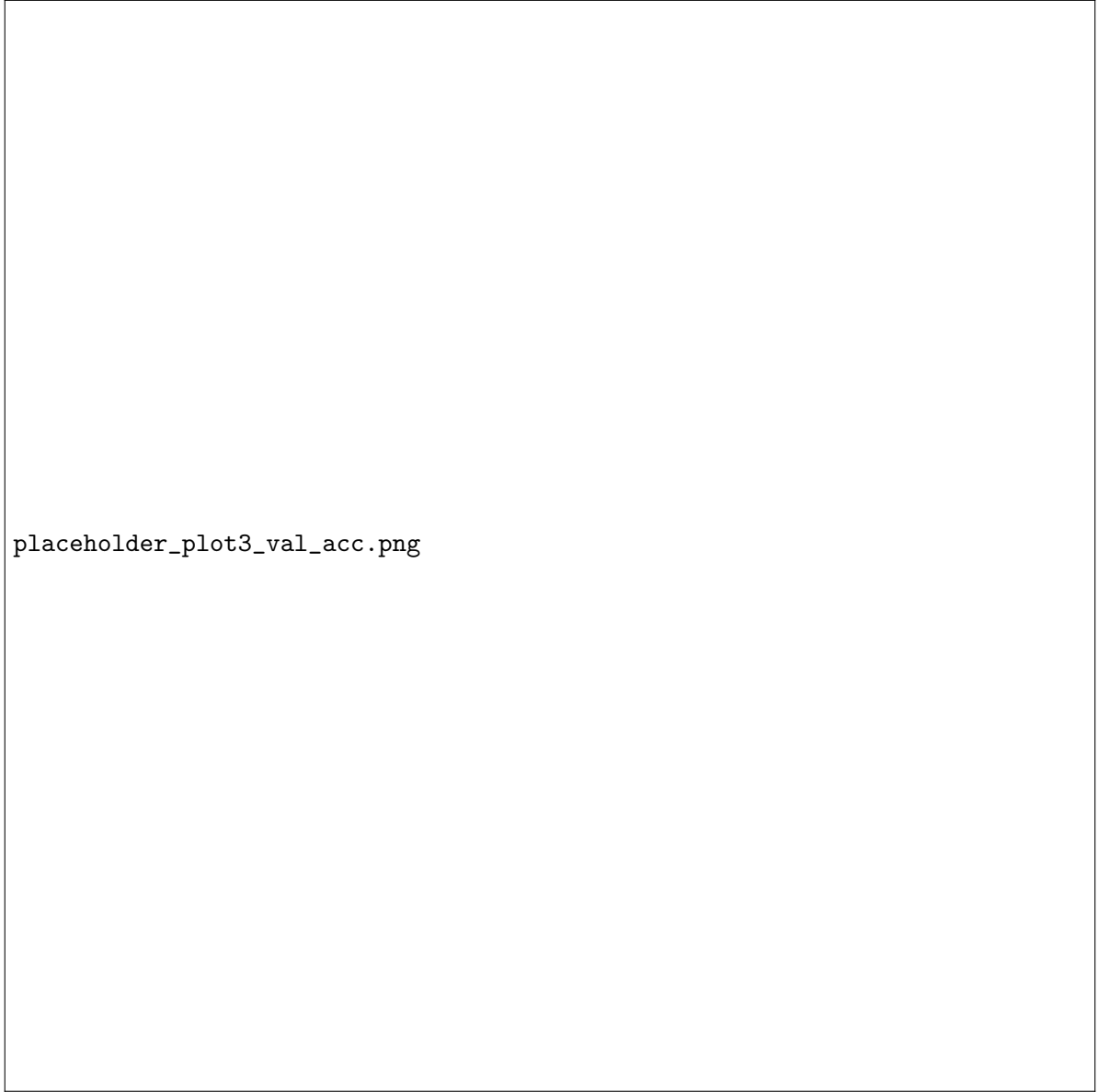


placeholder_plot2_train_acc.png

Figure 4: Training Accuracy vs. Epochs.

Interpretation: [Add your 2-3 line inference here]

3. Validation Accuracy



placeholder_plot3_val_acc.png

Figure 5: Validation Accuracy vs. Epochs.

Interpretation: [Add your 2-3 line inference here]

4. Training Loss



Figure 6: Training Loss vs. Epochs.

Interpretation: [Add your 2-3 line inference here]

5. Validation Loss

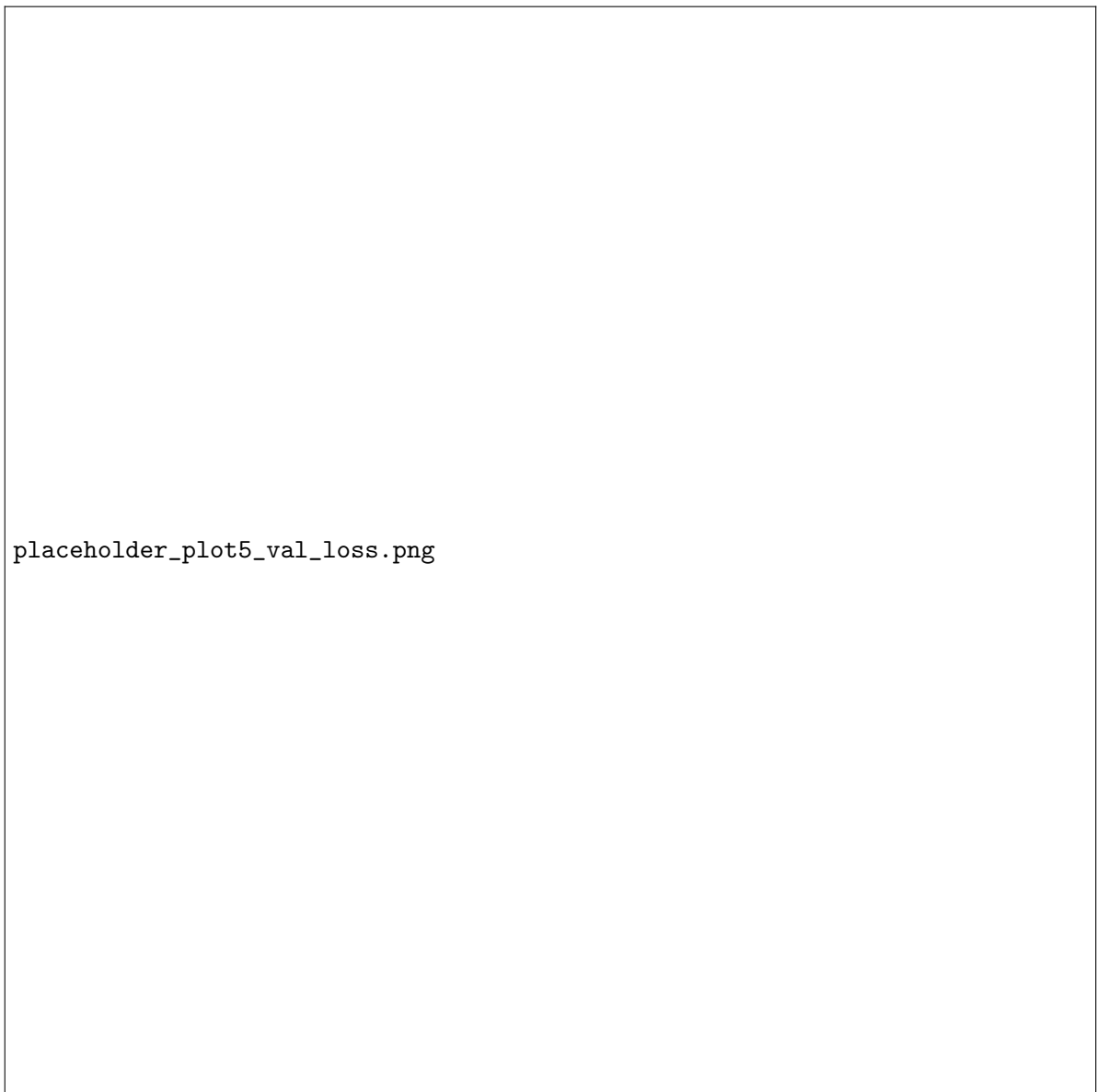


Figure 7: Validation Loss vs. Epochs.

Interpretation: [Add your 2-3 line inference here]

6. Confusion Matrix



Figure 8: Confusion Matrix on the test dataset.

Interpretation: [Add your 2-3 line inference here]

7. Misclassified Images (Optional)

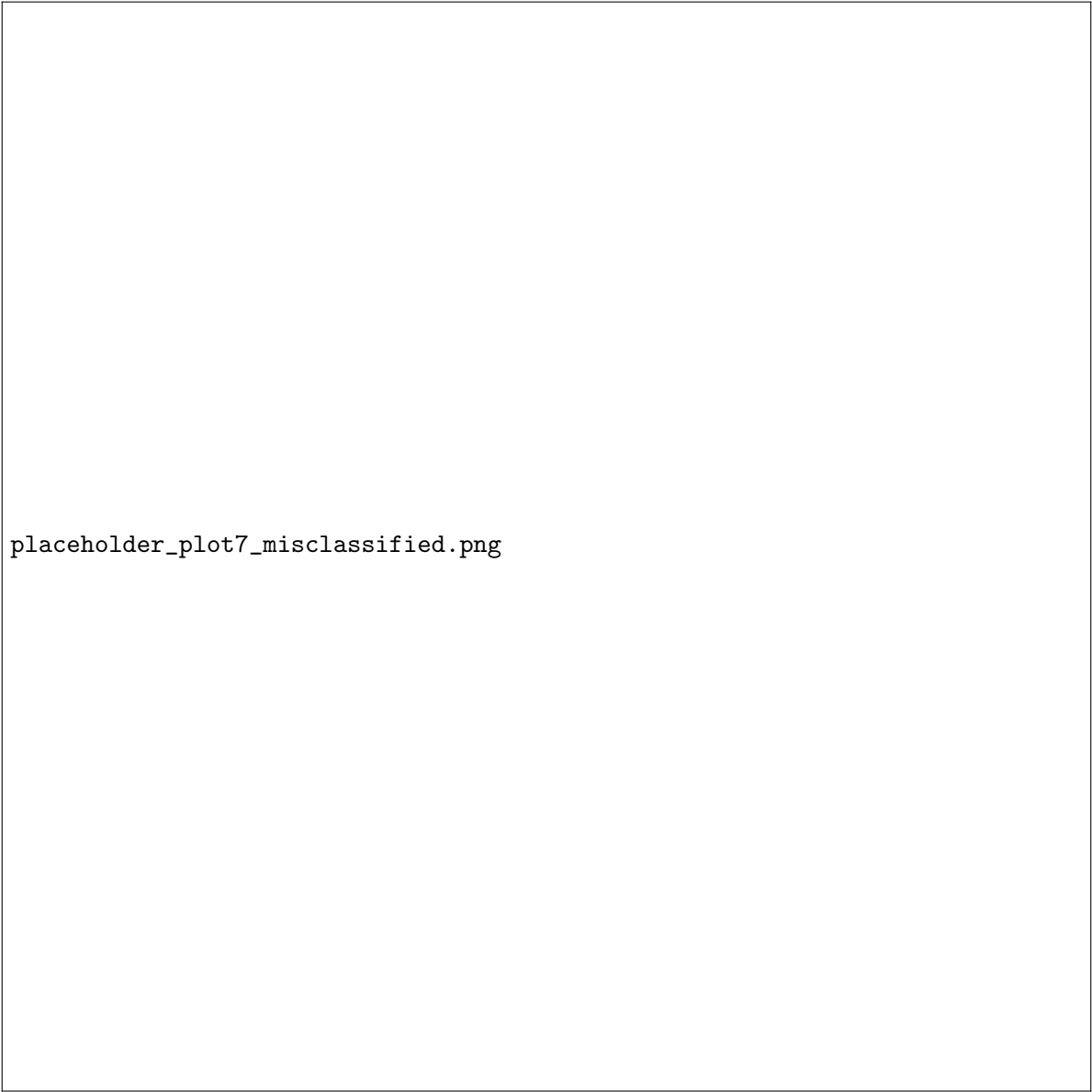


Figure 9: Examples of misclassified test images.

Interpretation: [Add your 2-3 line inference here]

19 Results

19.1 Performance Metrics

Metric	Value
Training Accuracy	
Testing Accuracy	
Precision	
Recall	
F1-score	
Training Time	
Total Parameters	

19.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5			
AlexNet			
VGG16			
GoogleNet			
ResNet50			

20 Discussion Questions

Answer the following questions.

1. Why is AlexNet considered a breakthrough in deep learning?
2. Why does VGG16 use only 3×3 convolution filters?
3. Explain the advantages of the Inception module.
4. What is the purpose of residual learning?
5. Differentiate LeNet and ResNet.
6. What is Transfer Learning?
7. Why is fine tuning required?
8. Explain the difference between dilated convolution and transpose convolution.
9. Why do pretrained models converge faster?
10. Compare the computational complexity of LeNet and ResNet.

21 Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.
4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

22 References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, Gradient-Based Learning Applied to Document Recognition, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, ImageNet Classification with Deep Convolutional Neural Networks, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, ICLR, 2015.
4. Christian Szegedy et al., Going Deeper with Convolutions, CVPR, 2015.

5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, Deep Residual Learning for Image Recognition, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, Deep Learning, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>